

Memòria del Projecte Final

Portada

- **Nom i cognoms:** Eduard Oliveras Guerrero
- **Estudis:** 2DAW
- **Centre:** IES Rafael Campalans
- **Títol del projecte:** Quickserve
- **Data:** 21 d'abril de 2026

\newpage

Índex

1. Introducció i objectius
2. Pla d'empresa
3. Anàlisi, disseny i construcció del projecte
4. Requeriments
5. Estructura de dades
6. Interfícies i experiència d'usuari
7. Seguretat i accés a dades
8. Còpies de seguretat
9. Problemes durant el desenvolupament
10. Seguiment diari
11. Conclusions
12. Bibliografia
13. Comentari personal
14. Annexos

\newpage

1. Introducció i objectius

1.1 Context del projecte

Quickserve és un projecte de digitalització orientat a resoldre una necessitat real en una rostisseria. El punt de partida és una operativa diària basada en paper per gestionar encàrrecs i part de les comandes. Aquest model manual funciona quan el volum de feina és baix, però en franges de màxima activitat provoca diversos problemes: duplicitat de tasques, pèrdua d'informació, dificultat per consultar l'estat de les comandes i risc d'errors humans.

La finalitat del projecte és construir una aplicació capaç de transformar aquest flux manual en un flux digital traçable, estable i adaptat a l'entorn de treball real. En lloc de plantejar una solució genèrica de TPV, Quickserve se centra en la manera concreta de treballar del negoci i en les necessitats d'un equip de mida reduïda.

1.2 Descripció general de la solució

Quickserve es basa en una arquitectura web amb backend Laravel i base de dades MySQL. El sistema incorpora una API amb autenticació per token i cobreix funcionalitats centrals de l'operativa:

- autenticació d'usuaris,
- consulta de catàleg,
- registre de vendes,
- gestió d'encàrrecs pendents,
- cobrament i cancel·lació d'encàrrecs,
- gestió administrativa de productes, categories, treballadors i comandes.

A nivell d'ús, la solució està pensada per respondre ràpidament en entorn de servei, evitant fluxos complexos i prioritzant la simplicitat operativa.

1.3 Motivació

La motivació del projecte és directament professional i vivencial. L'autor treballa en una rostisseria on l'ús de paper continua sent habitual per registrar encàrrecs. Aquest fet ha permès identificar de primera mà els punts de fricció del procés actual i traduir-los en objectius de millora.

A diferència d'un projecte plantejat només des d'un supòsit teòric, Quickserve neix amb una orientació pràctica: solucionar un problema que es produeix cada setmana, en un context real i amb usuaris reals.

1.4 Objectiu general

Dissenyar i implementar una aplicació de gestió de comandes i encàrrecs per a rostisseria que substitueixi el flux manual en paper per un flux digital més eficient, fiable i mantenible.

1.5 Objectius específics

1. Permetre registrar vendes i encàrrecs des d'una interfície clara i ràpida.
2. Garantir coherència de dades entre comandes, línies de detall i estoc.
3. Fer possible el seguiment d'encàrrecs pendents del dia en temps real.
4. Facilitar tasques administratives (catàleg, treballadors i consultes de comandes).
5. Assegurar una base tècnica preparada per evolucions futures.

1.6 Abast inicial i evolució de l'abast

L'abast inicial del projecte era limitar-se a un mòdul d'encàrrecs per tauleta. Durant les primeres setmanes de desenvolupament es va detectar una restricció crítica: no era viable integrar aquesta part amb l'aplicació ja existent al negoci.

Davant d'aquesta limitació, es va redefinir l'estratègia i es va optar per desenvolupar una aplicació més completa, capaç d'assumir una part substancial de la gestió diària. Aquesta evolució ha implicat un increment de complexitat (model de dades, fluxos de cobrament, seguretat, administració), però també ha multiplicat la utilitat real del projecte.

1.7 Alternatives considerades

Les alternatives reals valorades van ser:

- mantenir el sistema en paper,

- construir una solució pròpia.

Es descarta continuar només amb paper perquè no resol problemes de traçabilitat ni redueix errors en moments de càrrega alta. Es tria desenvolupar una eina pròpia per adaptar-se millor al flux real i conservar control total de l'evolució funcional.

1.8 Tecnologies triades i justificació

Backend i lògica de negoci: PHP 8.2 + Laravel 12.

- Permet estructurar el projecte en capes clares (rutes, controladors, models, serveis).
- Disposa d'eines integrades per validació, ORM, migrations i seguretat.
- Facilita mantenibilitat i velocitat de desenvolupament.

Base de dades: MySQL.

- Tecnologia robusta i àmpliament utilitzada.
- Bona integració amb Laravel.
- Suficient per al volum i tipus de dades del projecte.

Autenticació: Laravel Sanctum.

- Solució simple i eficient per API token-based.
- Adequada per clients web i aplicacions Flutter.

Client multiplataforma complementari: Flutter.

- Possibilita consum de la mateixa API des de web/escriptori/mòbil.
- Redueix duplicació de lògica entre plataformes.

2. Pla d'empresa

2.1 Estudi de mercat

La digitalització en petits negocis de restauració avança, però encara hi ha molts establiments que operen amb processos manuals o amb eines poc adaptades al seu cas concret. En negocis de barri, els factors crítics solen ser:

- rapidesa operativa,
- corba d'aprenentatge baixa,
- cost assumible,
- fiabilitat en moments de màxima demanda.

Quickserve se situa en aquest espai: una eina funcionalment ajustada, orientada a operativa real i sense costos de llicència.

2.2 Públic objectiu

- Rostisseries de mida petita o mitjana.
- Negocis de menjar preparat amb comandes per franja horària.
- Equips reduïts que necessiten simplicitat i rapidesa.

2.3 Proposta de valor

1. Digitalització de processos que habitualment es fan en paper.
2. Millor visibilitat de comandes i encàrrecs pendents.
3. Reducció d'errors derivats de gestió manual.
4. Possibilitat d'adaptar funcionalitats al negoci concret.

2.4 Anàlisi D.A.F.O.

Debilitats

- Projecte desenvolupat en context acadèmic i amb recursos limitats.
- Dependència inicial d'un únic desenvolupador.
- Encara sense procés formal de desplegament en producció continuada.

Amenaces

- Existència de solucions comercials consolidades.
- Resistència al canvi en equips acostumats al paper.
- Risc d'adopció parcial si no hi ha formació adequada.

Fortaleses

- Neix d'un problema real validat en entorn real.
- Ajust alt al flux operatiu concret de la rostisseria.
- Control total sobre evolució i millores.
- Cost de llicències nul.

Oportunitats

- Tendència general de digitalització de pimes.
- Escalabilitat del model a negocis similars.
- Possibilitat d'afegir mòduls de reporting i analítica.

2.5 Pressupost de maquinari

Maquinari necessari en escenari de desplegament:

- PC amb pantalla tàctil (x2): 900 EUR/unitat -> 1.800 EUR
- Tauleta (x1): 250 EUR -> 250 EUR
- Impressora de tiquets (x1): 180 EUR -> 180 EUR

Cost total estimat de maquinari: 2.230 EUR.

Inversió addicional real necessària: 250 EUR (la tauleta), ja que la resta ja està disponible al negoci.

2.6 Pressupost de software

El projecte s'ha desenvolupat amb eines gratuïtes i de codi obert.

- Laravel: 0 EUR
- MySQL: 0 EUR

- Flutter: 0 EUR
- GitHub (ús base): 0 EUR

Cost de llicències de software: 0 EUR.

2.7 Cost de dedicació

Per quantificar l'esforç invertit, s'estima:

- 180 hores de treball tècnic
- valoració estimada: 15 EUR/h

Cost teòric de dedicació: 2.700 EUR.

Aquest import representa cost d'oportunitat, no despesa directa a tercers.

2.8 Finançament

El projecte és autofinançat. Les despeses actuals i futures associades al desenvolupament o millores són assumides per l'autor.

2.9 Viabilitat inicial

Amb una inversió addicional reduïda i llicències inexistents, la viabilitat econòmica inicial és favorable. El principal factor de cost no és tecnològic, sinó de temps de desenvolupament i millora contínua.

3. Anàlisi, disseny i construcció del projecte

3.1 Metodologia de treball

S'ha utilitzat una metodologia iterativa i incremental:

1. definició de tasques curtes,
2. implementació progressiva,
3. proves funcionals,
4. correcció i refinament,
5. nova iteració.

Aquest enfocament ha permès adaptar-se a canvis d'abast sense bloquejar el progrés.

3.2 Fases i cronograma

Fase	Període estimat	Període real	Resultat
Ideació i planificació	Desembre	Desembre	Definició problema/abast inicial
Desenvolupament base	Gener	Gener	Primera versió operativa
Validació i proves	Gener-Abril	Gener-Abril	Correccions contínues
Consolidació i ampliació	Febrer-Abril	Febrer-Abril	Solució de gestió integral

3.3 Temporització mensual narrada

Desembre

- Anàlisi del problema real.
- Definició de requisits mínims inicials.
- Estudi de viabilitat tecnològica.

Gener

- Implementació de la base backend.
- Primeres funcionalitats de comandes.
- Inici de validació en casos d'ús reals.

Febrer

- Revisió d'abast i ampliació del model.
- Desenvolupament de fluxos d'encàrrecs més complets.
- Millores de consistència en dades.

Març

- Consolidació del mòdul d'administració.
- Ajustos de fluxos de cobrament/cancel·lació.
- Afinació de comportament en casos límit.

Abril

- Tancament funcional.
- Revisió de robustesa i qualitat de la solució.
- Preparació de memòria i documentació final.

3.4 Planificació i seguiment

Jira

- Organització de tasques per blocs funcionals.
- Seguiment de l'estat i prioritats.
- Visió global del progrés setmanal.

GitHub

- Traçabilitat completa de canvis en codi.
- Historial de commits com a evidència d'evolució.
- Sincronització remota del repositori (push).

3.5 Recursos utilitzats

- 1 desenvolupador.
- Entorn real de negoci per validar decisions.
- Equipament existent: 2 PCs tàctils + impressora.
- Tauleta addicional.

3.6 Gestió de riscos tècnics

Riscos identificats:

- inconsistències de dades en fluxos de comanda,
- errors d'estoc,
- augment d'abast no previst,
- dependència de validació manual.

Mesures aplicades:

- validacions estrictes d'entrada,
 - transaccions de BD,
 - proves iteratives,
 - modularització progressiva.
-

4. Requeriments

4.1 Àmbit i camp

Projecte de digitalització per restauració de proximitat amb focus en operativa de comandes/encàrrecs.

4.2 Requeriments funcionals

RF-01. El sistema ha de permetre autenticar usuaris.

RF-02. El sistema ha de permetre consultar l'usuari autenticat.

RF-03. El sistema ha de permetre tancar sessió de forma segura.

RF-04. El sistema ha de mostrar catàleg de categories i productes actius.

RF-05. El sistema ha de permetre crear una venda directa amb múltiples línies.

RF-06. El sistema ha de permetre crear encàrrecs amb nom de client i hora.

RF-07. El sistema ha de permetre consultar encàrrecs pendents del dia.

RF-08. El sistema ha de permetre cobrar encàrrecs pendents.

RF-09. El sistema ha de permetre cancel·lar encàrrecs.

RF-10. El sistema ha d'actualitzar estoc segons operacions realitzades.

RF-11. El sistema ha de generar numeració fiscal en comandes cobrades.

RF-12. El sistema ha de permetre gestió administrativa de categories.

RF-13. El sistema ha de permetre gestió administrativa de productes.

RF-14. El sistema ha de permetre pujada d'imatges de producte.

RF-15. El sistema ha de permetre gestió administrativa de treballadors.

RF-16. El sistema ha de permetre consulta detallada de comandes.

4.3 Requeriments no funcionals

RNF-01. Seguretat per token amb Sanctum.

RNF-02. Validació server-side de dades d'entrada.

RNF-03. Integritat de dades amb transaccions en processos crítics.

RNF-04. Temps de resposta adequat a fluxos de TPV.

RNF-05. Usabilitat orientada a personal no tècnic.

RNF-06. Mantenibilitat per arquitectura modular.

RNF-07. Escalabilitat funcional per mòduls futurs.

RNF-08. Disponibilitat operativa en horari de servei.

RNF-09. Traçabilitat tècnica de canvis via Git.

4.4 Casos d'ús principals

- UC-01 Iniciar sessió.
- UC-02 Consultar catàleg.
- UC-03 Crear venda.
- UC-04 Crear encàrrec.
- UC-05 Consultar encàrrecs pendents.
- UC-06 Cobrar encàrrec.
- UC-07 Cancel·lar encàrrec.
- UC-08 Gestionar productes.
- UC-09 Gestionar categories.
- UC-10 Gestionar treballadors.

4.5 Fitxa de cas d'ús (exemple extens)

UC-06 Cobrar encàrrec pendent

- **Actor principal:** Personal de venda.
- **Precondicions:** Usuari autenticat; encàrrec existent en estat pendent.
- **Disparador:** L'usuari selecciona "Cobrar" sobre un encàrrec.
- **Flux principal:**
 1. El sistema mostra dades de l'encàrrec.
 2. L'usuari selecciona mètode de pagament.
 3. El sistema valida dades.
 4. El sistema actualitza estat a pagat.
 5. El sistema assigna numeració fiscal.
 6. El sistema retorna confirmació.
- **Flux alternatiu A:** Encàrrec no existent -> error controlat.
- **Flux alternatiu B:** Dada invàlida -> resposta 422 i no modifica dades.

- **Postcondicions:** Encàrrec cobrat i coherent a nivell de dades.

4.6 Diagrames a incloure

- diagrama de casos d'ús general,
 - diagrama d'activitat de venda,
 - diagrama d'activitat d'encàrrec,
 - diagrama de classes principal,
 - diagrama de components.
-

5. Estructura de dades

5.1 Model conceptual

Entitats centrals:

- User
- Worker
- Category
- Product
- Order
- OrderItem

La separació entre **Order** (capçalera) i **OrderItem** (detall) permet una representació flexible de comandes amb qualsevol nombre de productes.

5.2 Relacions

- Worker (1) -> (N) Order
- Order (1) -> (N) OrderItem
- Product (1) -> (N) OrderItem
- Product (N) <-> (N) Category

5.3 Normalització i consistència

El disseny evita redundàncies innecessàries i garanteix consistència amb claus foranes. Operacions sensibles es protegeixen amb transaccions.

5.4 Decisions específiques de domini

- **is_preorder** per distingir comanda directa i encàrrec.
- **pickup_number** per identificar recollides del dia.
- **pickup_time** i **customer_name** per suportar entrega ordenada.
- **fiscal_full_number** per exigències de registre fiscal intern.

5.5 Estratègia d'estoc

En crear venda/encàrrec es descompta estoc segons línies de comanda. En cancel·lació d'encàrrec es restitueix estoc segons quantitats registrades. Aquesta lògica minimitza desviacions entre sistema i realitat.

6. Interfícies i experiència d'usuari

6.1 Principis de disseny d'interfície

1. rapidesa d'ús,
2. lectura clara,
3. minimització de passos,
4. baixa càrrega cognitiva.

6.2 Flux d'usuari: venda directa

- login,
- selecció productes,
- confirmació de pagament,
- registre final.

6.3 Flux d'usuari: encàrrec

- selecció productes,
- dades de recollida,
- registre pendent,
- consulta en llista de pendents,
- cobrament posterior.

6.4 Panell d'administració

Permet mantenir dades mestres i evitar dependència tècnica per tasques habituals de configuració.

6.5 Criteris d'usabilitat aplicats

- etiquetes clares,
- accions principals visibles,
- errors amb missatges comprensibles,
- ordenació de dades coherent.

7. Seguretat i accés a dades

7.1 Autenticació

Es fa servir autenticació per token amb Laravel Sanctum per protegir endpoints d'API.

7.2 Autorització i protecció de rutes

Les rutes de negoci només són accessibles per usuaris autenticats. Això impedeix accés anònim a dades sensibles.

7.3 Validació de dades

Totes les operacions crítiques (alta de comandes, cobrament, etc.) inclouen validació de camps per evitar dades incorrectes o incompletes.

7.4 Integritat transaccional

Els processos que poden afectar diverses taules es tracten dins transaccions de base de dades per evitar estats intermedis incoherents.

7.5 Registre i diagnosi

Els errors es registren per facilitar diagnosi i correcció d'incidències.

8. Còpies de seguretat

8.1 Estat actual

Actualment hi ha una còpia local del projecte en una carpeta externa.

8.2 Limitacions

- punt únic de fallada,
- manca de periodicitat formal,
- absència de separació codi/BD,
- recuperació no automatitzada.

8.3 Pla de millora proposat

1. còpia setmanal de codi,
 2. export de BD periòdic,
 3. còpia addicional en ubicació separada,
 4. procediment documentat de restauració.
-

9. Errors i problemes durant el desenvolupament

9.1 Problema principal: flux d'encàrrecs

Durant les primeres iteracions, la combinació de crear, cobrar i cancel·lar encàrrecs podia provocar inconsistències en estoc o en estat final de comanda.

9.2 Causes identificades

- validacions incompletes en alguns casos,
- lògica de transició d'estats millorable,
- necessitat de blindar operacions encadenades.

9.3 Solució implementada

- reforç de validacions,
- ús sistemàtic de transaccions,
- revisió de seqüència de canvis d'estat,
- proves iteratives amb casos reals.

9.4 Resultat

Flux d'encàrrecs estabilitzat i comportament més previsible en operativa diària.

9.5 Altres dificultats

- creixement d'abast no previst,
 - necessitat de prioritzar funcionalitats segons impacte real,
 - equilibri entre velocitat de desenvolupament i robustesa.
-

10. Seguiment diari

10.1 Eines de seguiment

- Jira per gestió funcional de tasques.
- GitHub per seguiment tècnic i historial de canvis.

10.2 Dietari resumit mensual

Desembre: definició del problema, planificació i enfoc.

Gener: construcció de base funcional i primeres proves.

Febrer: ampliació d'abast i refactor de fluxos.

Març: consolidació de mòduls i estabilització.

Abril: tancament de funcionalitats i documentació.

10.3 Dietari detallat i traçabilitat temporal

El seguiment detallat del projecte no es presenta en format plantilla, sinó com a registre real de sessions de treball agrupades per jornades amb impacte funcional. Aquest dietari s'ha consolidat al capítol 22, on s'explica l'evolució del projecte des del diagnòstic inicial fins al tancament final, incloent decisions d'abast, implementacions crítiques, incidències i validacions de regressió.

Per evitar duplicacions innecessàries dins la memòria, aquest apartat actua com a pont de lectura:

- **Traça cronològica completa:** veure el capítol 22 ([Dietari del desenvolupament](#), 18 entrades narratives).
- **Validació funcional de cada iteració:** veure el capítol 23 ([Annex de proves funcionals complet](#)).
- **Relació requisits-implementació:** veure el capítol 18 ([Matriu de traçabilitat](#)).

Aquesta estructura permet mantenir el document coherent: el capítol 10 resumeix, el capítol 22 argumenta el procés de treball i el capítol 23 aporta evidència de comportament del sistema.

11. Conclusions

Quickserve compleix l'objectiu de substituir una part crítica de la gestió manual en paper per una operativa digital més fiable i eficient. El projecte destaca per la seva orientació real: no és una prova conceptual, sinó una resposta a una necessitat quotidiana del negoci.

El canvi d'abast, inicialment vist com una dificultat, ha acabat sent una oportunitat per construir una solució més completa i útil. A nivell tècnic, el projecte consolida competències en arquitectura backend, API design, modelatge de dades, consistència transaccional i mantenibilitat.

A nivell professional, reforça la capacitat de prendre decisions, prioritzar i adaptar-se a canvis reals durant el desenvolupament.

12. Bibliografia

- [Documentació oficial de Laravel](#)
- [Documentació de Laravel Sanctum](#)
- [Documentació oficial de PHP](#)
- [Documentació oficial de MySQL](#)
- [Documentació oficial de Flutter](#)

13. Comentari personal

Aquest projecte ha estat especialment rellevant perquè parteix d'un problema viscut en primera persona. La motivació principal no ha estat només complir una entrega acadèmica, sinó construir una eina que pugui ser útil en un entorn real.

La part més valuosa del procés ha estat aprendre a ajustar l'abast quan les condicions canvien, mantenint alhora qualitat tècnica i orientació pràctica. En aquest sentit, Quickserve representa tant un aprenentatge tecnològic com una experiència de resolució de problemes de negoci.

14. Annexos de suport tècnic i evidència

Aquest bloc recull material de suport per reforçar la lectura acadèmica i documentar de manera explícita decisions, pantalles, proves i traçabilitat.

Annex A. Guió de captures comentades (flux operatiu complet)

La versió final de la memòria incorpora entre 16 i 24 captures. A continuació es defineix el guió mínim comentat (12 captures clau), amb el contingut explicatiu que acompanya cada imatge:

1. Login

- Objectiu: validar l'accés segur d'usuari i inici de sessió operatiu.
- Accions: introduir credencials, enviar formulari, gestionar error d'autenticació.
- Decisió UX: formulari net, missatge de credencial incorrecta comprensible i visible.

2. Catàleg

- Objectiu: mostrar productes venibles amb lectura ràpida en entorn de servei.
- Accions: navegar per categories, veure preu i estat de producte.
- Decisió UX: prioritat visual de producte i preu, evitant sobrecàrrega de text.

3. Cistella

- Objectiu: construir la comanda abans de confirmar pagament.
- Accions: afegir/eliminar línies, modificar quantitats, veure subtotal.
- Decisió UX: controls directes per minimitzar passos en hora punta.

4. Creació de venda

- Objectiu: tancar una comanda immediata amb persistència correcta.
- Accions: seleccionar mètode de pagament, confirmar venda.
- Decisió UX: confirmació clara i retorn ràpid d'estat.

5. Creació d'encàrrec

- Objectiu: registrar una comanda pendent amb dades de recollida.
- Accions: activar mode encàrrec, informar hora i client.
- Decisió UX: camps opcionals contextuais només quan `is_preorder=true`.

6. Llista de pendents

- Objectiu: consultar encàrrecs del dia en estat pendent.
- Accions: ordenar per hora, localitzar número de recollida.
- Decisió UX: lectura en llista compacta i orientada a acció.

7. Cobrament d'encàrrec

- Objectiu: convertir pendent a pagat mantenint coherència de total.
- Accions: cobrar, assignar mètode de pagament, confirmar estat final.
- Decisió UX: botó d'acció principal únic i confirmació final inequívoca.

8. Cancel·lació d'encàrrec

- Objectiu: revertir una comanda pendent sense inconsciències de dades.
- Accions: cancel·lar, confirmar acció, validar desaparició de pendents.
- Decisió UX: protecció davant accions destructives amb confirmació explícita.

9. Gestió de categories

- Objectiu: mantenir famílies de producte des de l'àrea admin.
- Accions: alta, edició i baixa amb validació d'unicitat.
- Decisió UX: formularis curts i feedback immediat de validació.

10. Gestió de productes

- Objectiu: governar catàleg viu (preu, estoc, actiu/inactiu).
- Accions: crear, editar, desactivar, associar categories.
- Decisió UX: separar camps obligatoris d'opcionals per reduir errors.

11. Gestió de treballadors

- Objectiu: administrar equip i control bàsic per PIN.
- Accions: alta/edició, activació/desactivació, comprovació de format PIN.
- Decisió UX: avisos clars en validacions de 4 dígit.

12. Detall de comanda

- Objectiu: auditar comanda completa (capçalera + línies + fiscal).
- Accions: consulta de línies i imports, verificació de número fiscal.
- Decisió UX: estructura jeràrquica per entendre ràpidament la comanda.

Annex B. Diagrames UML i lectura tècnica

Aquest annex ha d'incloure els diagrames, però també una interpretació curta de cada un:

Diagrama de casos d'ús

Cada diagrama incorpora una nota de 8-12 línies explicant què aporta al disseny i quina decisió concreta va ajudar a prendre.

Annex C. Evidències de seguiment (Jira + GitHub)

Aquest apartat acredita que el projecte s'ha construït de forma iterativa i traçable:

- export de Jira per sprint o blocs funcionals,
- captures del tauler amb estats (To Do, In Progress, Done),
- captures d'historial de commits rellevants,
- relació entre milestone funcional i entrega tècnica.

Exemple de traçabilitat:

Milestone	Evidència Jira	Evidència GitHub	Resultat funcional
Flux base de venda	Tasques ORD-01 a ORD-05 tancades	commits de creació POST /orders	venda simple i múltiple operativa
Flux d'encàrrec	Tasques PRE-01 a PRE-04	commits de pending, charge, cancel	cicle complet d'encàrrec disponible
Mòdul admin	Tasques ADM-01 a ADM-08	commits CRUD + dashboard	manteniment de dades i KPI operatius

Annex D. Contracte d'API (extracte representatiu)

La descripció completa d'endpoints es troba al capítol 17. Aquest annex en mostra una versió operativa amb exemples:

1. **POST /auth/login**
 - Entrada mínima: email, password, device_name.
 - Sortida esperada: token d'accés + dades bàsiques d'usuari.
 - Error habitual: credencials invàlides (422).
2. **GET /catalog**
 - Entrada: token vàlid.
 - Sortida esperada: categories i productes actius amb metadades de venda.
 - Error habitual: no autenticat (401).
3. **POST /orders**
 - Entrada: worker_id, items[], payment_method, is_preorder.
 - Sortida esperada: comanda creada amb línies i totals coherents.
 - Errors habituals: camps obligatoris absents (422), producte inexistent (422).
4. **POST /orders/{id}/charge**
 - Entrada: id d'encàrrec pendent + mètode pagament (+ opcional bossa).
 - Sortida esperada: canvi a Pagat + numeració fiscal.
 - Error habitual: encàrrec no pendent o no existent.
5. **POST /orders/{id}/cancel**
 - Entrada: id d'encàrrec pendent.
 - Sortida esperada: cancel·lació i restitució d'estoc.
 - Error habitual: id invàlid o estat incompatible.

Annex E. Proves funcionals

El catàleg complet de proves es documenta al capítol 23 (45 casos + 3 escenaris integrats). Aquest annex en destaca conclusions de qualitat:

- cobertura dels fluxos crítics de negoci (venda, encàrrec, cobrament, cancel·lació),
- validació d'errors de negoci i de validació (401/422),
- comprovació de consistència d'estoc en escenaris de reversió,
- verificació de mòdul administratiu i panell KPI.

Indicadors resum de validació

Bloc de proves	Casos executats	Estat general	Observació
Autenticació i seguretat	3	Validat	protecció de rutes validada
Operativa de comandes	18	Validat	coherència totals/estats confirmada
Administració	17	Validat	CRUD i filtres funcionals
KPI i consultes	7	Validat	estructura i càlcul consistents

Annex F. Pla de millora prioritzat (12 mesos)

Per facilitar evolució real del producte, es proposa un roadmap en tres fases:

Fase 1 (0-3 mesos): fiabilitat operativa

- automatització de backup codi + BD,
- prova periòdica de restauració,
- verificació de smoke test pre i post desplegament.

Fase 2 (3-6 mesos): governança funcional

- rols i permisos més granulars (admin, encarregat, venda),
- registre d'accions sensibles (auditoria bàsica),
- millora de textos d'error orientats a usuari final.

Fase 3 (6-12 mesos): valor analític i escalabilitat

- informes comparatius setmanals i mensuals,
- KPI avançats de producte/hores punta,
- estudi d'integració amb sistemes externs o extensió multiestabliment.

Annex G. Registre d'incidències reals i resolució aplicada

Aquest annex recull incidències representatives observades durant el desenvolupament i la validació funcional. L'objectiu és demostrar capacitat de diagnosi i correcció, no només implementació de funcionalitats.

ID	Context	Síntoma	Causa identificada	Acció correctiva	Estat
----	---------	---------	--------------------	------------------	-------

ID	Context	Síntoma	Causa identificada	Acció correctiva	Estat
INC-01	Creació de venda múltiple	Diferència puntual en total entre client i servidor	Càlcul parcial fet al client amb valors no normalitzats	Recalcul total al backend com a font de veritat	Resolta
INC-02	Alta d'encàrrec	Duplicat ocasional de pickup_number en proves manuals seguides	Lògica incremental insuficientment protegida en seqüència curta	Ajust de càlcul incremental diari i validació addicional	Resolta
INC-03	Cobrament amb bossa	Error quan el producte bossa no existia o estava inactiu	Falta de validació específica del producte auxiliar	Validació dedicada (bag_product_id) i missatge 422 explícit	Resolta
INC-04	Cancel·lació d'encàrrec	Restitució d'estoc incorrecta en un cas amb producte especial	Tractament incomplet de quantitats decimals	Revisió de càlcul i cobertura del cas de 0.5 unitats	Resolta
INC-05	Històric admin amb filtres	Resultats inconsistents combinant text + data + estat	Ordre d'aplicació de filtres no homogeni	Refactor de query amb when encadenat i proves de combinació	Resolta
INC-06	Upload imatges	Fallada en primera execució en entorn net	Directorí de destinació no creat	Creació automàtica de directorí abans de persistir fitxer	Resolta
INC-07	Validacions API	Missatges 422 massa genèrics en alguns endpoints	Missatges per defecte no contextualitzats	Millora de missatges de validació orientats a operativa	Resolta

Patrons detectats i aprenentatge

La revisió d'incidències va posar en evidència tres patrons útils per a projectes similars:

1. els errors de coherència solen aparèixer en fluxos encadenats (crear -> cobrar -> cancel·lar), no en operacions aïllades;
2. la robustesa augmenta quan les regles de negoci es centralitzen al backend i no es reparteixen entre client i servidor;
3. la qualitat percebuda per l'usuari millora molt quan els errors són explicatius i accionables.

Mesures preventives incorporades

Per reduir recurrència d'incidències, es van consolidar aquestes pràctiques:

- ús de transaccions en operacions crítiques de comanda,
- validació exhaustiva d'entrades abans de persistir,
- proves de regressió després de qualsevol canvi en fluxos d'encàrrec,

- revisió de casos límit de domini (productes especials, quantitats decimals, productes auxiliars).

Annex H. Decisions tècniques descartades i justificació

Documentar el que no s'ha implementat també és rellevant davant tribunal, perquè mostra criteri de prioritització i maduresa de disseny.

Decisió valorada	Alternativa descartada	Motiu principal de descart	Decisió final adoptada
Model de persistència	NoSQL documental per comandes	No aportava avantatge clar en relacions i consultes necessàries	Model relacional MySQL amb claus foranes
Gestió d'estats	Estat calculat només des de client	Risc alt d'inconsistència i manipulació	Estat de negoci calculat i validat a backend
Arquitectura backend	Monòlit sense separació de capes	Mantenibilitat baixa en creixement d'abast	Estructura Laravel per rutes/controladors/models/serveis
Identificació d'encàrrec	Taula separada per preorders	Duplicava lògica i incrementava complexitat de manteniment	Unificació a <code>orders</code> amb <code>is_preorder</code> i camps contextuais
Gestió d'errors	Missatges genèrics per codi HTTP	Poc útil en operativa real de servei	Missatges específics de validació i negoci
Control de permisos	RBAC complet des de la primera versió	Cost alt de desenvolupament per al calendari del TFC	Autenticació token + PIN en accions sensibles, amb RBAC com evolució

Impacte d'aquestes decisions en el projecte

Les decisions adoptades van permetre concentrar esforç en el nucli de valor (venda i encàrrecs) i reduir risc tècnic en fases primerenques. A més, van facilitar una evolució progressiva: primer assegurar consistència i usabilitat bàsica, després ampliar governança i analítica.

Decisions posposades per versió futura

No es van eliminar del roadmap, sinó que es van ajornar de manera explícita:

- rols i permisos granulars (RBAC complet),
- quadres de comandament comparatius més avançats,
- automatització completa de cicles de desplegament i restauració.

Aquest enfocament evita sobreenginyeria inicial i manté una línia de creixement coherent amb les necessitats reals del negoci.

15. Arquitectura tècnica detallada

15.1 Visió general d'arquitectura

Quickserve segueix un patró de capes clar orientat a API:

1. **Capa d'entrada (routing):** definició d'endpoints a `routes/api.php`.
2. **Capa d'aplicació (controllers):** orquestració de casos d'ús a `AuthController`, `CatalogController`, `OrderController` i `AdminController`.
3. **Capa de domini/persistència (models Eloquent):** representació d'entitats (`Order`, `OrderItem`, `Product`, `Category`, `Worker`, `User`) i relacions.
4. **Capa de serveis:** lògica especialitzada com assignació de numeració fiscal.
5. **Capa de dades (MySQL):** persistència relacional amb integritat via claus foranes i migracions versionades.

Aquest disseny separa responsabilitats i facilita manteniment, proves i evolució funcional.

15.2 Rutes i mòduls d'API

El fitxer `routes/api.php` organitza l'API en blocs:

- **Públic:** `GET /ping`, `POST /auth/login`.
- **Protegit (`auth:sanctum`):**
 - `GET /auth/me`, `POST /auth/logout`
 - `GET /catalog`
 - `GET /workers`
 - `POST /orders`
 - `GET /orders/pending`
 - `POST /orders/{id}/charge`
 - `GET /orders/{id}/details`
 - `POST /orders/{id}/cancel`
- **Administració (`/admin`):**
 - verificació PIN,
 - dashboard i KPI,
 - CRUD categories,
 - CRUD productes,
 - upload d'imatges,
 - CRUD treballadors,
 - consulta/històric de comandes.

L'estructura respon a una lògica de responsabilitat: operativa de venda i encàrrecs separada d'administració avançada.

15.3 Lògica de comandes i consistència

El mètode `store` de `OrderController` és un punt crític i incorpora:

- validació estricta de camps,
- transacció de base de dades per encapsular tota l'operació,
- creació de capçalera de comanda,

- creació de línies `order_items`,
- ajust automàtic d'estoc,
- assignació de numeració fiscal en vendes cobrades,
- resposta estructurada per al client.

L'ús de transaccions redueix risc d'estats intermedis en cas d'error (p. ex. capçalera creada però línies no guardades).

15.4 Flux d'encàrrecs pendents

Els encàrrecs es modelen amb:

- `is_preorder = true`,
- `status = Pendent`,
- `pickup_number` incremental diari,
- `pickup_time` i `customer_name` opcionals.

Accions disponibles:

1. alta d'encàrrec,
2. consulta de pendents del dia,
3. cobrament (canvi a estat pagat),
4. cancel·lació (restitució d'estoc i eliminació de comanda).

Aquest flux resol una necessitat clau del negoci: separar comandes immediates d'encàrrecs amb recollida posterior.

15.5 Mòdul administratiu

`AdminController` aporta funcionalitats de gestió i anàlisi:

- KPI de caixa i rendiment diari/mensual,
- top productes,
- hores punta,
- top productes per dia de setmana,
- CRUD complet d'entitats mestres,
- cerca i filtres avançats d'històric de comandes.

Aquesta capa converteix Quickserve en una eina no només transaccional sinó també de suport a decisions operatives.

16. Diccionari de dades i model relacional ampliat

16.1 Taula `users`

- `id` (PK)
- `name`
- `email` (únic)
- `password`
- `role` (afegit en migració posterior, valor per defecte `worker`)

- timestamps

Funció: gestió d'usuaris autenticats del sistema.

16.2 Taula **workers**

- **id** (PK)
- **name**
- **pin** (nullable, 4 dígits en validacions de CRUD)
- **active** (boolean)
- soft deletes
- timestamps

Funció: representa personal operatiu associat a comandes i control intern.

16.3 Taula **categories**

- **id** (PK)
- **name** (únic)
- **color**
- soft deletes
- timestamps

Funció: agrupar productes per famílies de venda.

16.4 Taula **products**

- **id** (PK)
- **name**
- **price** (decimal)
- **stock** (decimal nullable)
- **active** (boolean)
- **is_gluten_free** (boolean)
- **description** (nullable)
- **image_path** (nullable)
- soft deletes
- timestamps

Funció: catàleg venible a TPV i encàrrecs.

16.5 Taula pivot **category_product**

- **id** (PK)
- **product_id** (FK -> products)
- **category_id** (FK -> categories)
- timestamps

Funció: relació N..N entre productes i categories.

16.6 Taula **orders**

- `id` (PK)
- `worker_id` (FK -> workers)
- `total_price` (decimal)
- `payment_method` (Efectiu, Targeta, etc.)
- `status` (Pagat, Pendent, ...)
- `is_preorder` (boolean)
- `pickup_number` (nullable)
- `pickup_time` (nullable)
- `customer_name` (nullable)
- `fiscal_series`, `fiscal_sequence`, `fiscal_full_number` (nullable)
- timestamps

Funció: capçalera de venda/encàrrec.

16.7 Taula `order_items`

- `id` (PK)
- `order_id` (FK -> orders)
- `product_id` (FK -> products)
- `quantity` (integer)
- `price_at_sale` (decimal)
- `notes` (nullable)
- timestamps

Funció: detall de línies de comanda.

16.8 Taula `invoice_sequences`

- `id` (PK)
- `series` (únic)
- `next_number`
- timestamps

Funció: suport de numeració fiscal seqüencial.

16.9 Taula `personal_access_tokens`

Persistència de tokens Sanctum per autenticació API.

16.10 Regles d'integritat clau

1. Una comanda no pot existir sense treballador (`worker_id`).
2. Un `order_item` sempre pertany a una comanda existent.
3. Un `order_item` sempre referencia un producte existent.
4. L'eliminació de capçalera elimina en cascada les línies associades.

17. Especificació funcional endpoint per endpoint

17.1 Autenticació

POST /auth/login

- **Objectiu:** autenticar usuari i retornar token.
- **Entrada:** email, password, device_name.
- **Sortida correcta:** token + dades bàsiques d'usuari.
- **Sortida d'error:** credencials incorrectes.

GET /auth/me

- **Objectiu:** recuperar usuari autenticat actual.

POST /auth/logout

- **Objectiu:** invalidar token actiu.

17.2 Catàleg i dades bàsiques

GET /catalog

- Retorna categories i productes actius.
- Inclou preu, estoc, categories associades i imatge.

GET /workers

- Retorna treballadors actius ordenats per nom.

17.3 Operativa de comandes

POST /orders

- Crea venda o encàrrec segons **is_preorder**.
- Guarda capçalera + línies.
- Actualitza estoc.
- Assigna numeració fiscal si és venda cobrada.

GET /orders/pending

- Llista encàrrecs del dia en estat pendent.

POST /orders/{id}/charge

- Cobra encàrrec pendent.
- Pot afegir bosses i actualitzar total.
- Assigna numeració fiscal.

GET /orders/{id}/details

- Retorna detall complet de comanda amb línies i treballador.

POST /orders/{id}/cancel

- Cancel·la encàrrec i restaura estoc.

17.4 Administració

POST /admin/verify-pin

- Valida PIN de treballador per accions sensibles.

GET /admin/dashboard

- Retorna KPI de venda, caixa, top productes, hores punta i distribucions.

CRUD categories/productes/treballadors

- Altes, canvis i baixes lògiques/físiques segons entitat.

GET /admin/orders

- Històric paginat amb filtres per estat, mètode de pagament, dates, treballador i text de cerca.

GET /admin/orders/{id}

- Vista detallada d'una comanda de l'històric.

18. Matriu de traçabilitat de requisits i implementació

Requisit	Endpoint / Component	Evidència de compliment
RF-01 Autenticació	/auth/login , Sanctum	Token retornat i rutes protegides
RF-04 Catàleg	/catalog , CatalogController	Productes/categoríes actius en resposta
RF-05 Crear venda	POST /orders	Comanda Pagat + línies + estoc
RF-06 Crear encàrrec	POST /orders amb is_preorder=true	Comanda Pendent + pickup_number
RF-07 Pendants del dia	/orders/pending	Consulta filtrada per data i estat
RF-08 Cobrar encàrrec	/orders/{id}/charge	Canvi d'estat + numeració fiscal
RF-09 Cancel·lar encàrrec	/orders/{id}/cancel	Eliminació + restauració d'estoc
RF-11 Numeració fiscal	Servei fiscal + camps orders	fiscal_full_number en resposta
RF-13 Gestió productes	/admin/products CRUD	Altes/modificacions/baixes en catàleg
RF-15 Gestió treballadors	/admin/workers CRUD	Control d'actius i PIN
RF-16 Històric comandes	/admin/orders	Paginació + filtres + detall

19. Pla de proves complet

19.1 Estratègia

Les proves es divideixen en:

1. proves d'autenticació,
2. proves de flux principal de venda,
3. proves de flux d'encàrrecs,
4. proves d'administració,
5. proves de consistència de dades,
6. proves de regressió després de canvis crítics.

19.2 Casos de prova suggerits (ampliables)

ID	Cas	Passos resumits	Resultat esperat
CP-01	Login correcte	enviar credencials vàlides	200 + token
CP-02	Login incorrecte	password erroni	422 error controlat
CP-03	Consultar catàleg	GET /catalog autenticat	categories + productes
CP-04	Crear venda simple	1 producte, is_preorder=false	201 + venda pagada
CP-05	Crear venda múltiple	3 línies de producte	totals i línies coherents
CP-06	Crear encàrrec	is_preorder=true	estat Pendent
CP-07	Llistar pendants	GET /orders/pending	inclou encàrrec creat
CP-08	Cobrar encàrrec	POST /charge	estat Pagat
CP-09	Cancel·lar encàrrec	POST /cancel	baixa i estoc restituit
CP-10	Cobrament amb bosses	bag_count > 0	total actualitzat
CP-11	Alta producte admin	POST /admin/products	producte disponible
CP-12	Edició producte admin	PUT /admin/products/{id}	canvis persistits
CP-13	Baixa producte admin	DELETE /admin/products/{id}	no visible en flux normal

ID	Cas	Passos resumits	Resultat esperat
CP-14	Alta treballador	<code>POST /admin/workers</code>	nou treballador actiu
CP-15	Filtres històric	<code>GET /admin/orders</code> amb paràmetres	resultats filtrats
CP-16	Dashboard KPI	<code>GET /admin/dashboard</code>	estructura KPI correcta
CP-17	Token invàlid	endpoint protegit sense token	401/accés denegat
CP-18	Dades invàlides comanda	camp obligatori absent	422 sense canvis BD
CP-19	Cerca per client	filtre <code>search</code>	llista coincideix amb criteri
CP-20	Concurrencia bàsica	dues operacions seguides sobre encàrrec	consistència d'estat

19.3 Criteris d'acceptació

- 100% dels casos crítics del flux de venda/encàrrec en estat validat.
- 0 errors de consistència en estoc després de seqüències de crear-cobrar-cancel·lar.
- cap endpoint protegit accessible sense autenticació.

20. Pla de desplegament i operació

20.1 Entorn local de desenvolupament

- servidor Laravel en local,
- base de dades MySQL,
- client web/mòbil connectat a API,
- control de versions amb GitHub.

20.2 Procés de pas a producció

1. revisió de configuració `.env`,
2. còpia de seguretat prèvia,
3. migracions controlades,
4. prova de smoke test funcional,
5. validació amb usuari final,
6. monitoratge inicial durant primera setmana.

20.3 Riscos operatius i mitigacions

- **Risc:** caiguda de base de dades. **Mitigació:** backups regulars i prova de restauració.
- **Risc:** error humà en ús inicial. **Mitigació:** guia curta i formació de 30-45 min.

- **Risc:** regressió en canvis futurs. **Mitigació:** comprovació de proves abans de publicar.
-

21. Pla de qualitat i manteniment

21.1 Qualitat de codi

- separació de responsabilitats,
- noms coherents,
- validació d'entrada centralitzada a controladors,
- ús de relacions Eloquent per simplicitat de consulta.

21.2 Manteniment correctiu

Flux de manteniment:

1. detectar incidència,
2. reproduir cas,
3. corregir en branca específica,
4. executar proves de regressió,
5. desplegar canvi corregit.

21.3 Manteniment evolutiu

Línies d'evolució prioritàries:

- automatització de backup,
 - millor reporting,
 - permisos més granulars,
 - possible multitenancy si s'escala a més establiments.
-

22. Dietari del desenvolupament

Aquest dietari recull l'evolució del projecte en jornades de treball substancials. En lloc de puntualitzar cada petita acció com a entrada independent, s'han agrupat les tasques per sessió real de feina, ja que en una mateixa jornada sovint es van abordar diversos fronts encadenats (disseny, implementació, prova i correcció). Cada entrada inclou la data o rang de dates, les hores aproximades dedicades i una descripció narrativa del que es va fer, dels entrebancs trobats i de l'estat amb què es va tancar la sessió.

Entrada 01 — Diagnòstic del problema i definició del MVP (2–5 de desembre de 2025, ~3,5 h)

Les primeres sessions del projecte van ser d'observació més que no pas de codi. Durant un torn de servei real a la rostitseria es va parlar atenció al flux de comandes i encàrrecs tal com s'executa habitualment: notes en paper, anotacions a mà, dubtes recurrents sobre què estava pendent, què ja s'havia cobrat i quines comandes havien de sortir a quina hora. D'aquesta observació directa en va sortir una llista inicial de problemes palpables, com la pèrdua ocasional de notes, la dificultat de saber l'estat real d'un encàrrec sense preguntar-ho a algú, i la impossibilitat de revisar el que havia passat en una franja concreta del dia. Amb aquella diagnosi sobre la taula, la segona sessió es va dedicar a filtrar idees i a definir un MVP realista: no es tractava de construir "tot el que seria desitjable", sinó el que resol·lia la major fricció operativa amb

menys esforç. Va quedar clar que el nucli havia de ser vendes directes i encàrrecs pendents, i la resta podia esperar. Aquesta priorització, feta molt d'hora, va estalviar hores de disseny especulatiu més endavant.

Entrada 02 — Elecció d'stack i model de dades inicial (9–14 de desembre de 2025, ~4,5 h)

Amb el problema acotat, la decisió tècnica següent va ser triar l'stack. Es va fer una comparativa ràpida entre opcions de backend (principalment Laravel vs alternatives en Node) i de base de dades, valorant maduresa de l'ecosistema, facilitat per estructurar el projecte en capes i compatibilitat futura amb un client mòbil. La combinació Laravel 12 + MySQL + autenticació per token amb Sanctum va sortir guanyadora per productivitat i per quantitat d'eines integrades. La jornada del 14 es va centrar en el model de dades. Sobre paper, es van dibuixar les entitats bàsiques i les seves relacions, i ja es va prendre una decisió de disseny important: separar clarament la capçalera de comanda (**orders**) del detall de línies (**order_items**). Aquesta separació és estàndard però no trivial, perquè obria la porta a representar qualsevol mida de comanda sense haver de tocar l'esquema. També es va començar a perfilar que una futura entitat de "treballador" havia de viure per sobre de l'usuari autenticat, perquè qui valida o registra no sempre és qui té la sessió oberta.

Entrada 03 — Setup de projecte, migracions i relacions Eloquent (20 de desembre de 2025 i 8–10 de gener de 2026, ~9,5 h)

Aquesta va ser una de les sessions fundacionals del projecte. Primer es va muntar tot l'entorn: repositori Git, esquelet de Laravel, configuració d'entorn local, primeres dependències i arrencada verificada. Amb el projecte ja responent a un **GET /ping**, es va atacar la capa de persistència amb calma. Les migracions de **categories**, **products**, **workers**, **orders** i **order_items** es van escriure revisant amb lupa claus foranes, cascades i tipus de dada, perquè una migració mal plantejada aquí es paga en hores de refactor molt més endavant. Tot seguit, es van crear els models Eloquent corresponents i les relacions: **hasMany** entre comanda i línies, **belongsToMany** cap al treballador, i una pivot **category_product** per suportar el cas real en què un mateix producte pot pertànyer a més d'una categoria. Hi va haver petits xocs a l'hora de mantenir coherència entre la migració de producte (amb camps com **is_gluten_free**, **description** o **image_path**) i el **fillable** del model, però es van resoldre en la mateixa sessió amb una ronda de proves a través de **tinker**. En acabar, l'esquema relacional ja estava viu i consultable.

Entrada 04 — Autenticació amb Sanctum i endpoint de catàleg (12–14 de gener de 2026, ~5 h)

Amb la base de dades ja posada, calia protegir-la. Es va implementar l'autenticació per token amb Laravel Sanctum, incloent-hi **POST /auth/login**, **GET /auth/me** i **POST /auth/logout**. La configuració inicial de Sanctum va demanar una mica de polida (especialment al voltant del **device_name** i de la caducitat raonable del token per a un entorn de TPV), però un cop ajustada, tots els endpoints de negoci van quedar darrere del middleware **auth:sanctum**. Un cop verificat que un token no vàlid donava 401 i que un token vàlid permetia entrar, es va publicar el primer endpoint útil: **GET /catalog**. Aquest endpoint retorna categories i productes actius amb les seves relacions, i se'n va definir el format de resposta pensant en un client Flutter que havia de pintar graells. Es va ajustar el mapeig perquè la mateixa resposta servís també per a web, evitant duplicar lògica per plataforma.

Entrada 05 — Creació de vendes amb línies i totals (16–19 de gener de 2026, ~7 h)

Aquesta va ser la primera jornada llarga centrada en el flux de venda. El punt crític era el **POST /orders**: un endpoint que, aparentment senzill, ha d'orquestrar diverses operacions coherents. Es va començar per

la versió bàsica, que creava la capçalera amb dades de pagament i de treballador, i es va anar afegint validació estricta de cada camp. Seguidament, es va estendre per persistir les línies de comanda a `order_items`, vinculant cada una al producte corresponent i guardant el preu en el moment de la venda (`price_at_sale`), perquè canvis futurs de tarifa no havien de reescriure l'històric. Aquí van sortir els típics embolics de totals: petites diferències entre el subtotal calculat en client i el recalculat en servidor. La decisió va ser clara: el servidor sempre és la font de veritat i recalcula tot a partir de les línies. En acabar la jornada ja es podien registrar vendes complexes de diverses línies amb totals coherents.

Entrada 06 — Control d'estoc i transaccions de base de dades (21–24 de gener de 2026, ~5,5 h)

Havent validat que les vendes es guardaven bé, tocava connectar-les amb l'estoc. El decrement automàtic de `stock` en registrar una venda va ser ràpid de posar, però va obligar a pensar diversos casos: productes amb `stock` nul (que no es controlen), productes amb decimals (anticipant el cas del mig pollastre) i productes donats de baixa a mig flux. També va ser aquesta jornada la que va consolidar una pràctica que marcaria la resta del projecte: tot el bloc de creació de comanda (capçalera + línies + ajust d'estoc + numeració fiscal quan procedeix) ha de viure dins una transacció de BD. Aquesta decisió es va traduir en un `DB::transaction` ben delimitat al `store` del `OrderController`, amb un tractament explícit d'errors per fer rollback en cas de fallada a qualsevol pas. Amb això es va eliminar la possibilitat de quedar-se amb capçaleres sense línies o amb estocs descompensats si una operació es trencava a mig fer.

Entrada 07 — Disseny i alta d'encàrrecs amb número de recollida (28–31 de gener de 2026, ~7 h)

L'arrencada del mòdul d'encàrrecs va ser una de les parts més rellevants del projecte, perquè representa la traducció directa del problema real observat al desembre. Primer es va dissenyar com diferenciar una venda immediata d'un encàrrec diferit sense haver de crear una taula nova: la decisió va ser incorporar `is_preorder` com a flag i afegir `pickup_number`, `pickup_time` i `customer_name` a `orders`, tots nullable perquè no apliquen a vendes directes. La sessió del 31 es va dedicar a implementar l'alta d'encàrrecs reutilitzant el mateix `POST /orders`, ramificant el comportament pel flag. La part més delicada va ser la numeració de recollida: havia de ser incremental dins del dia i començar de nou l'endemà, sense col·lidir amb encàrrecs d'altres jornades. També es va decidir que l'estat inicial fos `Pendent`, deixant per a un endpoint posterior el canvi a `Pagat`. En acabar, ja es podien crear encàrrecs identificables per número de recollida i amb dades de client.

Entrada 08 — Llista de pendents i cobrament d'encàrrecs (4–8 de febrer de 2026, ~6 h)

Els encàrrecs sense una manera d'enumerar-los i de cobrar-los no servien de res operativament. La primera part d'aquesta jornada va ser muntar `GET /orders/pending`, que filtra per `is_preorder = true`, `status = Pendent` i data del dia actual, i ordena per hora de recollida. Aquí va caldre decidir què es consideraven "encàrrecs del dia": es va triar la data de creació, que en l'entorn real coincideix pràcticament sempre amb la recollida. La part de cobrament (`POST /orders/{id}/charge`) va ser més elaborada. No n'hi havia prou a canviar `status` a `Pagat`: calia garantir que, si l'encàrrec s'havia creat amb un total determinat, el cobrament pogués ajustar-lo si el client afegia productes al moment i, alhora, deixar assignada la numeració fiscal corresponent. Es van lligar totes les peces perquè un cobrament fos una operació única, consistent i reversible en cas d'error. Aquest punt va ser clau per fer que el flux d'encàrrecs fos realment usable en servei.

Entrada 09 — Bossa en el cobrament i cancel·lació d'encàrrec (11–14 de febrer de 2026, ~7 h)

El 11 de febrer es va afegir una funcionalitat aparentment petita però molt demandada en l'operativa real: poder incloure bosses en el moment de cobrar un encàrrec. Això va significar introduir els paràmetres `bag_count` i `bag_product_id`, afegir una línia automàtica al detall de la comanda amb el preu de la bossa recuperat del producte corresponent i recalculat el total. El cas més desagradable va aparèixer quan, per dades mal mestrejades, el producte "bossa" no existia o estava inactiu: en lloc de trencar l'operació, la validació retorna un 422 clar sense tocar la comanda. La sessió del 14 va atacar l'altre costat del mateix flux: la cancel·lació d'encàrrec (`POST /orders/{id}/cancel`). Aquesta operació havia d'eliminar la comanda pendent i, el més important, retornar l'estoc al seu estat previ línia a línia. Es van provar casos amb diversos productes, amb productes sense control d'estoc i amb quantitats variades, i tot va quedar encapsulat també dins una transacció per evitar que una cancel·lació a mig camí deixés el sistema en un estat ambigu.

Entrada 10 — Cas del mig pollastre i replantejament d'abast (18–22 de febrer de 2026, ~4,5 h)

El 18 es va atacar un cas molt propi del negoci: el mig pollastre. A nivell d'interfície es venia com un producte diferent, però a nivell d'estoc afectava el mateix producte base amb un decrement de 0,5 unitats. Això va obligar a revisar la lògica de stock per tolerar decimals tant en vendes com en cancel·lacions. És el típic detall que un desenvolupament genèric no cobriria mai, però que en l'operativa diària apareix constantment. El 22, amb el mòdul d'encàrrecs ja robust, es va fer una revisió d'abast més àmplia. Es va valorar fins on havia d'arribar el projecte i es va confirmar una decisió de pes: l'eina existent al negoci no es podia integrar, així que Quickserve s'havia de convertir en una solució més completa, capaç de cobrir gestió administrativa (catàleg, personal, consultes) a més dels fluxos de venda. Aquesta sessió no va produir codi, però sí el roadmap funcional que va guiar tot el bloc de març.

Entrada 11 — Administració de categories i productes (26 de febrer i 1 de març de 2026, ~7 h)

La primera peça del bloc administratiu va ser el CRUD de categories. Es van muntar els endpoints d'alta, edició i baixa, amb validació d'unicitat de nom i amb un comptador de productes associats a cada categoria pensat per evitar eliminacions que deixessin orfes productes actius. Un cop tancades les categories, es va atacar el CRUD de productes, molt més ric: camps bàsics (nom, preu), opcionals (descripció, `is_gluten_free`), control d'activació i sincronització de la relació N..N amb categories mitjançant `sync`. Aquí va aparèixer una particularitat: en editar un producte, calia decidir si es permetia canviar-ne les categories de cop o si es bloquejava. Es va triar permetre-ho i registrar el canvi, perquè en un negoci real els productes canvien de família amb certa freqüència (productes de temporada, per exemple). També es va afegir la baixa lògica (soft delete) per no perdre històric de vendes amb productes retirats.

Entrada 12 — Pujada d'imatges i administració de treballadors (5–9 de març de 2026, ~5,5 h)

El 5 de març es va implementar la pujada d'imatges de producte com a endpoint independent. Es va delimitar amb validació de tipus (`jpg`, `png`) i de mida raonable, i es va decidir que el nom final del fitxer no contingués l'original per evitar col·lisions o problemes de codificació. En la primera execució va sortir el petit entrebanc habitual: el directori de destinació no existia, així que el codi va haver de crear-lo si calia. Un cop provat amb diverses imatges, l'endpoint retornava `image_path` i ja es podia associar al producte. El 9 de març es va muntar el CRUD de treballadors: nom, PIN de 4 dígits quan procedeix, flag `active` i soft deletes. El PIN va demanar validació estricta (exactament 4 dígits numèrics) per evitar dades corruptes, i la desactivació d'un treballador havia de fer-lo desaparèixer de les llistes operatives sense eliminar el seu rastre en comandaments antigues. Amb això, el mòdul administratiu ja cobria totes les entitats mestres.

Entrada 13 — Històric de comandes filtrat i dashboard amb KPI (13–17 de març de 2026, ~8 h)

La jornada del 13 es va dedicar íntegrament al `GET /admin/orders`. No era un simple llistat: havia de suportar paginació, filtre per estat, per mètode de pagament, per rang de dates, per treballador, i una cerca lliure per nom de client o per número fiscal. Combinar tots aquests filtres de manera que l'`Eloquent` resultant fos llegible i prou eficient va demanar una mica de treball, amb `when` encadenats per aplicar només els filtres presents a la request. També es va afegir el detall (`GET /admin/orders/{id}`) perquè es pogués consultar qualsevol comanda amb les seves línies i el treballador assignat. El 17 es va atacar el `GET /admin/dashboard`: KPI de caixa diari, distribució per mètode de pagament, ticket mig, top de productes venuts, distribució per hores del dia (8h–15h) i top de productes per dia de la setmana. Va ser una sessió llarga perquè cadascuna d'aquestes mètriques demana una consulta específica ben pensada. En acabar, el panell oferia una visió de jornada ràpida i consumible des de qualsevol client autenticat.

Entrada 14 — Numeració fiscal automàtica i seqüències (22 de març de 2026, ~3,5 h)

Aquesta sessió es va centrar en un requisit més formal: assignar una numeració fiscal seqüencial a les comandes cobrades. Es va crear una taula `invoice_sequences` amb sèrie i `next_number`, i un servei dedicat a generar la següent numeració respectant idempotència (és a dir, si per qualsevol motiu el codi arriba dues vegades al mateix punt amb la mateixa comanda, no ha d'emetre un segon número). Els camps `fiscal_series`, `fiscal_sequence` i `fiscal_full_number` es van afegir a `orders` per deixar-hi el rastre. L'assignació es va integrar tant al `store` (quan la venda ja neix pagada) com al `charge` (quan un encàrrec passa a pagat). En acabar, qualsevol comanda pagada tenia una numeració estable, consultable des de l'històric i reutilitzable per a cerques.

Entrada 15 — Reforç de validacions i proves de regressió d'encàrrecs (29 de març i 4 d'abril de 2026, ~6,5 h)

El 29 es va fer una revisió sistemàtica de totes les validacions d'entrada. L'objectiu no era trobar errors greus sinó millorar la qualitat dels missatges de resposta: molts 422 eren genèrics i poc útils per al client, així que es van reformular per apuntar clarament al camp afectat i a la causa. Es va revisar, també, que cap endpoint deixés persistència parcial en cas de dada invàlida. El 4 d'abril es va muntar una bateria de proves manuals exhaustives sobre encàrrecs, seguint seqüències reals: crear amb diverses línies, aparcar, afegir bosses, cobrar, i també provar el camí de cancel·lació en diversos punts del flux. Va aparèixer un ajust menor en el càlcul quan una bossa es cobrava sobre un encàrrec sense línies addicionals, però es va corregir a la mateixa sessió. El flux principal va quedar validat sense regressions greus i amb molta més confiança per fer-lo servir en entorn real.

Entrada 16 — Estabilització de l'estoc i neteja final de codi (10–15 d'abril de 2026, ~5,5 h)

El 10 d'abril es va fer una última passada sobre el control d'estoc. S'havien detectat petits desajustos de precisió decimal quan es combinaven vendes amb mig pollastre i cancel·lacions posteriors. Es va normalitzar l'ús de tipus numèrics i es va homogeneïtzar com es fan els càlculs per evitar arrossegat d'errors de coma flotant. El 15 es va dedicar a neteja de codi: repàs dels controladors principals (`OrderController`, `AdminController`, `CatalogController`), simplificació d'algunes respostes JSON que havien anat creixent amb camps redundants i eliminació de codi mort heretat de proves inicials. Aquesta fase no afegia funcionalitat però deixava l'API preparada per ser documentada amb coherència i per suportar canvis futurs sense haver de descodificar cada vegada la intenció original.

Entrada 17 — Consolidació de documentació API i traçabilitat de requisits (18–20 d'abril de 2026, ~4 h)

El 18 es va atacar la documentació tècnica del contracte d'API: revisar cada endpoint, preparar exemples de request i response representatius i unificar la terminologia emprada a missatges i camps. Es van detectar petits desajustos menors (alguns textos en castellà barrejats amb català, algun camp que apareixia en alguns mapejos i no en d'altres), que es van corregir per deixar una superfície d'API consistent. El 20 es va fer l'exercici complementari: creuar la llista de requisits funcionals i no funcionals del capítol 4 amb la implementació real per construir la matriu de traçabilitat del capítol 18. Aquesta feina, més documental que tècnica, va confirmar que tots els requisits prioritaris tenien endpoint o component associat amb evidència de compliment, i va treure a la llum algun petit buit documental sense cap implicació funcional.

Entrada 18 — Preparació final del lliurament (21 d'abril de 2026, ~2 h)

La jornada final es va centrar en el tancament formal del projecte. Es va fer una última passada ortogràfica i d'estil sobre tota la memòria, es van revisar salts de capítol, encapçalaments i referències creuades, i es va validar que l'estructura seguís la pauta establerta al començament del document. No hi va haver incidències bloquejants. El repositori de codi es va deixar en un estat estable amb l'últim commit ben descrit, i la memòria en format Markdown preparada per ser exportada a PDF amb la maquetació establerta. El projecte Quickserve queda així tancat com a lliurament, però amb una base prou neta per seguir evolucionant en les línies futures descrites més endavant al document.

23. Annex de proves funcionals complet

Aquest annex recull proves representatives i coherents amb l'operativa real del projecte. S'ha prioritzat cobertura de processos crítics: vendes, aparcament de tiquets/encàrrecs, cobrament, cancel·lació, bossa, administració i estoc.

23.1 Convencions

- **Prioritat Alta:** impacte directe en venda/caixa o integritat de dades.
- **Prioritat Mitjana:** funcionalitats d'administració i consultes.
- **Prioritat Baixa:** millores de confort operatiu.

23.2 Casos de prova

ID	Prioritat	Cas de prova	Entrada resumida	Resultat esperat
PF-01	Alta	Login vàlid	usuari correcte	token vàlid i accés
PF-02	Alta	Login invàlid	password incorrecta	error 422 controlat
PF-03	Alta	Accés sense token	crida a endpoint protegit	error d'autorització
PF-04	Alta	Consulta catàleg	GET /catalog	categories i productes actius

ID	Prioritat	Cas de prova	Entrada resumida	Resultat esperat
PF-05	Alta	Crear venda simple	1 producte, pagat	comanda creada + stock decrementat
PF-06	Alta	Crear venda múltiple	4 línies de cistella	capçalera + ítems coherents
PF-07	Alta	Venda amb producte sense stock controlat	stock null	venda permesa sense error de decrement
PF-08	Alta	Crear encàrrec bàsic	<code>is_preorder=true</code>	estat pendent + pickup_number
PF-09	Alta	Crear encàrrec amb hora i client	<code>pickup_time,</code> <code>customer_name</code>	dades guardades correctament
PF-10	Alta	Llistar encàrrecs pendents	<code>GET /orders/pending</code>	només pendents del dia
PF-11	Alta	Cobrar encàrrec pendent	<code>POST /charge</code>	estat a pagat + factura
PF-12	Alta	Cobrar encàrrec amb bossa	<code>bag_count=1</code>	item bossa afegit + total incrementat
PF-13	Alta	Cobrar encàrrec amb múltiples bosses	<code>bag_count=3</code>	subtotal de bosses correcte
PF-14	Alta	Cobrar encàrrec amb producte bossa inexistent	id incorrecte	error 422 i cap canvi de comanda
PF-15	Alta	Cancel·lar encàrrec	<code>POST /cancel</code>	comanda eliminada i stock restituint
PF-16	Alta	Cancel·lar encàrrec ja cobrat	id no pendent	comportament controlat segons estat
PF-17	Alta	Cas mig pollastre en venda	producte especial	decrement de 0.5 sobre Pollastre
PF-18	Alta	Cas mig pollastre en cancel·lació	cancel·lar comanda amb mig pollastre	restitució de 0.5
PF-19	Alta	Validació de comanda incompleta	manca <code>worker_id</code>	error 422, sense persistència parcial
PF-20	Alta	Transacció fallida en creació	error forçat en ítem	rollback complet
PF-21	Mitjana	Alta categoria admin	nom + color	categoria creada

ID	Prioritat	Cas de prova	Entrada resumida	Resultat esperat
PF-22	Mitjana	Categoria duplicada	mateix nom	error de validació
PF-23	Mitjana	Edició categoria	canvi de nom/color	valors actualitzats
PF-24	Mitjana	Eliminació categoria	id existent	baixa aplicada
PF-25	Mitjana	Alta producte admin	camp obligatoris	producte creat i visible
PF-26	Mitjana	Editar producte admin	canvi preu/stock	dades actualitzades
PF-27	Mitjana	Producte inactiu	<code>active=false</code>	fora de flux de venda
PF-28	Mitjana	Pujar imatge vàlida	jpg/png < 4MB	path retornat
PF-29	Mitjana	Pujar imatge invàlida	fitxer no imatge	error validació
PF-30	Mitjana	Alta treballador amb PIN	nom + pin 4 dígits	treballador creat
PF-31	Mitjana	Alta treballador pin duplicat	pin existent	error validació
PF-32	Mitjana	Editar treballador	canvi nom/actiu	persistència correcta
PF-33	Mitjana	Desactivar treballador	<code>active=false</code>	no apareix en workers actius
PF-34	Mitjana	Verificar PIN admin correcte	pin existent	<code>ok=true</code>
PF-35	Mitjana	Verificar PIN admin incorrecte	pin erroni	<code>ok=false</code> + error
PF-36	Mitjana	Històric comandes paginat	<code>per_page=20</code>	meta correcta
PF-37	Mitjana	Filtre històric per estat	<code>status=Pagat</code>	només pagades
PF-38	Mitjana	Filtre per mètode pagament	<code>payment_method=Efectiu</code>	només efectiu
PF-39	Mitjana	Filtre per rang de dates	<code>from/to</code>	comandes dins rang

ID	Prioritat	Cas de prova	Entrada resumida	Resultat esperat
PF-40	Mitjana	Cerca per client o fiscal	<code>search=Carla</code>	coincidències rellevants
PF-41	Mitjana	Dashboard KPI estructura	<code>GET /admin/dashboard</code>	blocs kpi/caixa/top
PF-42	Mitjana	KPI ticket mig	dades de prova	càlcul consistent
PF-43	Mitjana	Top productes	històric de vendes	ordenació descendent
PF-44	Mitjana	Hores punta	comandes per hora	sèrie 8-15 completa
PF-45	Baixa	Ping API	<code>GET /ping</code>	servei <code>ok=true</code>

23.3 Proves de seqüència operativa completa

Escenari SO-01 (Servei estàndard)

1. Login treballador.
2. Crear venda de dos productes.
3. Verificar decrement d'estoc.
4. Crear encàrrec per a les 13:30.
5. Consultar pendents i localitzar encàrrec.
6. Cobrar encàrrec afegint 1 bossa.
7. Comprovar numeració fiscal assignada.
8. Revisar històric admin.

Resultat esperat: tota la seqüència es completa sense incoherències d'estat ni de totals.

Escenari SO-02 (Cancel·lació controlada)

1. Crear encàrrec amb 3 línies.
2. Verificar decrement d'estoc inicial.
3. Cancel·lar encàrrec abans de cobrament.
4. Comprovar restitució completa d'estoc.
5. Confirmar absència a pendents.

Resultat esperat: sistema coherent després de la reversió.

Escenari SO-03 (Administració completa)

1. Alta de categoria nova.
2. Alta de producte vinculat.
3. Edició de preu i stock.
4. Upload d'imatge.
5. Desactivar producte.

6. Verificar que no apareix al catàleg de venda.

Resultat esperat: cicle de vida de producte operatiu de punta a punta.

23.4 Conclusions de proves

- Els fluxos crítics de negoci (venda, encàrrec, cobrament i cancel·lació) són funcionals.
- La part administrativa cobreix manteniment de dades mestres amb validacions.
- El control d'estoc respon correctament en casos habituals i en escenaris de reversió.
- La funcionalitat d'incloure bossa en cobrament queda integrada i traçable.

24. Síntesi documental del projecte

Aquesta versió amplia substancialment la memòria amb dos blocs de gran pes documental:

1. **Dietari de desenvolupament (18 entrades narratives)** per justificar evolució temporal i dedicació amb prosa descriptiva per jornada de treball real.
2. **Annex de proves funcionals (45 casos + 3 escenaris integrats)** per demostrar validació real del sistema.

Amb aquests apartats, la memòria consolida la traçabilitat del projecte i integra evidències tècniques del desenvolupament (dietari narratiu, proves, annexos comentats i relació requisits-implementació).

25. Desenvolupament narratiu ampliat del projecte

25.1 Del problema real a la definició del producte

El punt de partida de Quickserve no és una idea abstracta ni una proposta genèrica de digitalització, sinó una necessitat observada de manera directa en un entorn de treball real. En una rostisseria, el ritme de servei en hores punta obliga a prendre decisions ràpides i a registrar informació de forma immediata. Quan aquest registre es fa principalment en paper, apareixen friccions que, en moments de molta activitat, es converteixen en problemes recurrents: papers que es perden, comandes amb notes difícils d'interpretar, dubtes sobre què està pendent i què ja està preparat, i manca de traçabilitat quan cal revisar què ha passat en una franja concreta del dia.

Aquesta realitat va condicionar completament la definició del producte. En lloc d'intentar fer un sistema molt ampli des del primer moment, es va optar per identificar quin era el nucli de valor operatiu: registrar comandes de manera fiable, gestionar encàrrecs pendents amb ordre i reduir errors humans en el flux de treball. Aquesta decisió és rellevant perquè determina tota l'arquitectura posterior. El projecte no es construeix per demostrar una tecnologia, sinó per resoldre una seqüència de microproblemes que, sumats, afecten la qualitat del servei.

Durant les primeres setmanes va quedar clar que la paraula clau no era només "digitalitzar", sinó "operativitzar". Digitalitzar un procés manual sense repensar-lo sovint només trasllada el problema de suport: del paper a la pantalla. Per això Quickserve es va plantejar com una eina de procés, no només com un repositori de dades. Això implica que cada pantalla, cada camp i cada validació havien de respondre a preguntes concretes: aquest pas és realment necessari?, aquesta dada s'utilitza en una decisió posterior?, aquest flux aguanta una situació de pressa real, amb interrupcions i múltiples tasques en paral·lel?

Aquesta manera de definir el projecte va tenir una conseqüència positiva: les funcionalitats van quedar alineades amb l'ús real i no amb una suposada "completitud" teòrica. En un TFC és habitual voler incloure moltes peces per demostrar amplitud tècnica, però en aquest cas la prioritat va ser construir una base robusta sobre els processos de més impacte diari. Aquesta orientació ha permès que el projecte tingui sentit tant acadèmic com pràctic.

25.2 Decisió d'arquitectura i impacte en la mantenibilitat

L'elecció de Laravel com a backend va estar molt influïda per la necessitat d'equilibri entre productivitat i estructura. En un projecte que evoluciona d'un abast inicial reduït a una solució més completa, és essencial que el codi pugui créixer sense perdre llegibilitat. Laravel facilita aquest creixement perquè separa responsabilitats de manera natural: rutes per definir contracte, controladors per orquestrar fluxos, models per encapsular dades i relacions, i serveis per a lògiques transversals que no han d'estar incrustades directament a la capa HTTP.

L'arquitectura basada en API també ha estat una decisió estratègica, no només tècnica. Definir la lògica de negoci darrere endpoints clars permet desacoblar la implementació interna de la presentació. Això ha fet possible mantenir una mateixa font de veritat per diferents clients i, alhora, documentar millor el comportament esperat del sistema. Quan el projecte va començar a créixer, aquesta separació va evitar que els canvis d'interfície impactessin directament en la coherència del model de dades.

Un aspecte especialment important ha estat la consistència transaccional en operacions de comanda. En un context de venda, les operacions no són trivials: crear capçalera, crear línies, ajustar estoc, assignar estats i, en alguns casos, completar numeració fiscal. Si qualsevol part falla i la resta es guarda parcialment, el sistema queda en un estat difícil de reparar manualment. Per això la transacció no és un detall d'implementació, sinó una decisió de qualitat de dades. Aquesta pràctica ha reduït el risc d'incoherències i ha aportat confiança en el flux principal.

També cal remarcar que la mantenibilitat no depèn només de l'arquitectura, sinó de la disciplina d'evolució. El fet de treballar amb Jira i GitHub ha ajudat a mantenir context de canvi: per què s'ha fet una modificació, quin problema resol i quina part del flux podia veure's afectada. Aquesta traçabilitat és especialment valuosa quan es revisen regressions o quan s'han de justificar decisions a la memòria.

25.3 Evolució d'abast: de mòdul puntual a sistema operatiu

L'evolució més significativa del projecte és el canvi d'abast. Inicialment es volia construir una peça concreta per a encàrrecs des de tauleta. Aquesta idea tenia sentit com a primera aproximació perquè atacava un dolor clar i permetia un desenvolupament ràpid. Tanmateix, en avançar es va evidenciar que una peça aïllada no resol realment el problema de fons, ja que no es podia connectar de manera efectiva amb l'aplicació que l'empresa feia servir en aquell moment.

Aquest punt va ser decisiu: o bé es mantenia un mòdul parcial amb utilitat limitada, o bé es feia el salt a una solució integrada amb capacitat real de substituir processos. Es va escollir la segona opció. Aquesta decisió va augmentar la càrrega de treball i la complexitat tècnica, però també va donar coherència al projecte. En termes de memòria, aquest canvi d'abast justifica gran part del desenvolupament realitzat entre febrer i abril i explica per què es van incorporar mòduls administratius, historials filtrables, control d'estat de comandes i mètriques operatives.

Treballar amb un abast viu ha estat també un exercici de gestió de prioritats. No tot allò desitjable es podia implementar en el temps disponible, de manera que calia decidir què era imprescindible per garantir valor de negoci immediat i què quedava com a línia futura. Aquesta capacitat de priorització és una competència clau en projectes reals i forma part de l'aprenentatge més rellevant del procés.

25.4 Disseny del flux de comandes i encàrrecs

El flux de comandes és el cor del sistema. En una venda directa, l'objectiu és minimitzar passos sense comprometre consistència. El sistema ha de permetre seleccionar productes, confirmar pagament i registrar la comanda en un temps curt, amb resposta clara per a la persona que està atenent. Aquesta rapidesa és necessària perquè el cost d'una interfície lenta o confusa no és només tecnològic; és operatiu i es tradueix en cues, interrupcions i estrès de l'equip.

El flux d'encàrrecs, en canvi, té una naturalesa diferent: no acaba en el moment del registre inicial. Quan una comanda queda pendent, el sistema ha de conservar estat, hora i context de recollida, i ha de permetre reprendre l'operació més tard per completar-ne el cobrament o, si cal, cancel·lar-la. Aquesta dualitat (instantani vs diferit) obliga a modelar bé els estats de negoci i a evitar ambigüitats. Quickserve ho resol diferenciant explícitament encàrrec i venda, i incorporant camps que permeten ordenar la recollida i identificar-la de manera fiable.

Un cas especialment representatiu és la inclusió de bossa en el moment de cobrament d'un encàrrec. Pot semblar un detall menor, però és un exemple perfecte de requisit real no previst inicialment que impacta en total, línies de comanda i coherència fiscal. Implementar-ho correctament va requerir ajustar validacions, càlculs i persistència per evitar desajustos. Aquest tipus de cas mostra que la qualitat d'un sistema de venda depèn molt de com tracta els petits casos operatius que es repeteixen cada dia.

25.5 Control d'estoc com a eix de fiabilitat

El control d'estoc ha estat una de les parts més sensibles del projecte perquè qualsevol error aquí es veu immediatament a l'operativa. Si el sistema descompta de més, es pot bloquejar una venda que en realitat era possible. Si descompta de menys, es creen expectatives incorrectes i es trasllada el problema al moment de preparació. Per això el tractament d'estoc s'ha abordat com una responsabilitat transversal associada als fluxos de venda, encàrrec i cancel·lació.

La lògica no és només aritmètica. També incorpora decisions de domini, com el cas del mig pollastre, on la quantitat afectada sobre el producte base no és una unitat sencera. Aquest tipus de particularitat és habitual en negocis reals i obliga a sortir d'una implementació genèrica. La memòria incorpora aquest cas perquè representa molt bé el tipus d'adaptació que converteix una solució estàndard en una eina útil per a un context concret.

Des del punt de vista de qualitat, l'objectiu del control d'estoc no és únicament que els números quadrin al final del dia, sinó que el sistema sigui previsible en qualsevol seqüència operativa habitual. Crear, cobrar, cancel·lar, editar producte, desactivar-lo: cada pas pot afectar disponibilitat i, per tant, la confiança en l'eina. Aquesta confiança és el factor decisiu perquè l'equip abandoni definitivament el paper.

25.6 Mòdul administratiu i valor de gestió

El mòdul administratiu s'ha incorporat per cobrir una necessitat sovint infravalorada: mantenir la base de dades viva sense dependència constant de desenvolupament. En un negoci petit, les dades mestres

canvien sovint: productes temporals, ajustos de preu, altes o baixes de personal, canvis de categories. Si qualsevol d'aquests canvis requereix tocar base de dades manualment o editar codi, el sistema es torna inviable a curt termini.

Per això Quickserve inclou CRUD de categories, productes i treballadors, a més d'un historial de comandes amb filtres i un dashboard amb KPI operatius. Aquest dashboard no pretén substituir eines de BI, però sí donar una lectura immediata de la jornada: total de caixa, distribució de mètodes de pagament, ticket mitjà, top productes i franges horàries amb més activitat. Aquestes dades tenen valor perquè permeten prendre decisions senzilles però útils, com ajustar preparació de producte o planificar reforços puntuals.

La part administrativa aporta també valor acadèmic a la memòria perquè mostra que el projecte no es queda en un flux únic de demostració, sinó que contempla governança mínima de dades i observabilitat funcional del servei.

25.7 Seguretat aplicada al context del projecte

La seguretat s'ha tractat des d'una perspectiva pragmàtica orientada al risc real del projecte. En primer lloc, s'ha implementat autenticació per token amb Sanctum per garantir que les rutes de negoci només siguin accessibles per usuaris autenticats. En segon lloc, s'ha reforçat la validació de dades d'entrada en operacions crítiques per evitar persistència d'informació incompleta o inconsistent.

A més, en funcionalitats d'administració s'ha incorporat verificació de PIN de treballador per a accions sensibles. Tot i que no és un sistema de permisos avançat de nivell empresarial, és un mecanisme coherent amb el context operatiu i amb el nivell de risc que es vol cobrir en aquesta fase del projecte. Aquesta aproximació incremental és intencionada: es prioritza desplegar una base segura i clara, i deixar com a evolució futura una capa de rols i permisos més granular.

Cal destacar que la seguretat efectiva no és només autenticació. També és consistència d'errors, control de casos límit i capacitat de recuperar-se de fallades sense corrompre dades. En aquest sentit, la combinació de validacions i transaccions és una part central de la proposta de qualitat.

25.8 Aprenentatges tècnics i metodològics

Un dels aprenentatges més importants ha estat entendre que desenvolupar en context real obliga a conviure amb incertesa. Les especificacions no són estàtiques, els fluxos canvien quan s'observen en pràctica i els requisits més rellevants sovint apareixen quan el sistema ja està en ús inicial. Això no és un problema, sinó una característica natural del desenvolupament orientat a negoci.

A nivell tècnic, el projecte ha reforçat competències en disseny d'API, modelat relacional, coherència transaccional i organització de codi mantenible. A nivell metodològic, ha consolidat la importància de traçabilitat, prioritjació i validació contínua. Aquest doble aprenentatge és clau perquè converteix un exercici acadèmic en una experiència propera a un projecte professional.

També ha quedat clar que la qualitat final d'un producte no depèn d'una única gran decisió, sinó de moltes decisions petites ben enllaçades: noms de camps coherents, respostes API consistents, validacions que protegeixen el flux, i una documentació que explica el perquè de cada elecció. La memòria extensa és, en aquest sentit, una oportunitat per fer visible aquesta feina de detall que sovint no es veu només mirant el codi.

25.9 Impacte operatiu i valor pràctic

L'impacte de Quickserve s'ha de valorar en termes de reducció de fricció operativa. Quan un sistema permet veure què està pendent, què ja s'ha cobrat i quin és l'estat d'una comanda en pocs segons, el benefici no és només tecnològic: millora la coordinació de l'equip, redueix interrupcions i minimitza errors que abans eren habituals en suport paper.

A més, el fet de tenir historial, filtres i mètriques bàsiques facilita revisar la jornada amb criteri. Això obre la porta a una cultura de millora contínua basada en dades, encara que sigui en una escala petita. En molts negocis, aquest pas és més transformador que incorporar funcionalitats molt avançades però poc utilitzades.

Per tot plegat, Quickserve es pot considerar un projecte amb valor aplicat real: resol un problema existent, s'adapta al context de treball i deixa una base tècnica preparada per créixer.

25.10 Línies futures amb justificació

Les línies futures no es plantegen com una llista genèrica, sinó com una evolució natural de la base actual. La primera prioritat és professionalitzar les còpies de seguretat, separant clarament còpia de codi i còpia de base de dades, i incorporant verificació periòdica de restauració. La segona és ampliar la capa de permisos per rols detallats, especialment si el sistema s'utilitza per més perfils amb diferents nivells de responsabilitat.

Una tercera línia és reforçar analítica operativa amb informes comparatius setmanals i mensuals. Amb les dades que ja es recullen, es podrien construir indicadors de productivitat i demanda que ajudessin a planificar millor compres i preparació. Finalment, una evolució de gran impacte seria consolidar un desplegament estable de producció amb processos més formals de publicació i verificació postdesplegament.

Aquestes línies futures mantenen la coherència amb el projecte: no busquen afegir complexitat gratuïta, sinó augmentar fiabilitat, control i utilitat real del sistema en el seu context d'ús.

26. Reflexió final ampliada

La construcció d'aquesta memòria extensa posa de manifest que el valor d'un projecte final no es troba només en el resultat visible, sinó en el procés de decisions que hi ha al darrere. En el cas de Quickserve, el recorregut ha estat especialment formatiu perquè ha obligat a combinar tres dimensions alhora: necessitat de negoci, qualitat tècnica i capacitat d'adaptació.

Des de la perspectiva acadèmica, el projecte demostra domini de competències rellevants del cicle: modelatge de dades, desenvolupament backend, integració API, validació, seguretat bàsica, mantenibilitat i documentació. Des de la perspectiva professional, demostra alguna cosa igualment important: capacitat d'escollar el context, detectar què aporta valor real i prioritzar sota restriccions de temps.

El valor d'una memòria tècnica es basa en la qualitat del contingut i en la claredat amb què es justifiquen les decisions. Per això s'ha explicat no només què s'ha implementat, sinó també per què s'ha fet així, quins problemes resol i quins compromisos s'han assumit en cada fase. Aquesta traçabilitat argumental és la que dona solidesa al document final.

En definitiva, Quickserve no és únicament una aplicació per gestionar comandes i encàrrecs. És el resultat d'un procés de maduració tècnica i metodològica orientat a resoldre un problema real de manera

pragmàtica, coherent i evolutiva.
